CO3-AT1 EXPERIMENTS

Name: ADHITHYA.S
Register Number: 192412352
Course Code: ITA0606
Course Name: MACHINE LEARNING

Experiment 1: Naïve Bayes Classifier on a Real-World Dataset

Aim

To implement a Naïve Bayes classifier on a real-world dataset, train it using different feature sets, evaluate performance using accuracy/precision/recall, analyze the effect of the feature-independence assumption, and compare results with and without Laplace smoothing.

Dataset

Wisconsin Breast Cancer Diagnostic dataset (scikit-learn built-in, load_breast_cancer): 569 samples, 30 numeric features describing cell nuclei characteristics, binary target (malignant / benign). Split 75% train / 25% test with stratification.

Theory

Naïve Bayes applies Bayes' theorem with the (naïve) assumption that features are conditionally independent given the class: $P(y|x_1 ,...,x_n) \propto P(y)\ \Pi\ P(x_i|y)$. GaussianNB models each numeric feature as Gaussian per class. For categorical/count-style features, Laplace (add-one) smoothing adds a small constant to every count so that a feature value never seen with a given class during training does not force its conditional probability — and hence the whole posterior — to zero.

Procedure / Code

```
from sklearn.datasets import load_breast_cancer
from sklearn.naive_bayes import GaussianNB, CategoricalNB
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.preprocessing import KBinsDiscretizer
from sklearn.metrics import accuracy_score, precision_score, recall_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

# (a) GaussianNB with different feature sets: all 30 / top-10 / top-5
gnb = GaussianNB().fit(X_train, y_train)
selector = SelectKBest(f_classif, k=10).fit(X_train, y_train)
gnb10 = GaussianNB().fit(selector.transform(X_train), y_train)

# (b) Laplace smoothing comparison on binned (categorical) features
kbins = KBinsDiscretizer(n_bins=8, encode='ordinal', strategy='quantile')
X_train_bin = kbins.fit_transform(X_train_small)
cnb_smooth   = CategoricalNB(alpha=1.0).fit(X_train_bin, y_train_small)
cnb_nosmooth = CategoricalNB(alpha=1e-10).fit(X_train_bin, y_train_small)
```

Results

Configuration	Accuracy	Precision	Recall
GaussianNB — all 30 features	0.9371	0.9263	0.9778

Configuration	Accuracy	Precision	Recall
GaussianNB — top-10 features (ANOVA F-test)	0.9301	0.9348	0.9556
GaussianNB — top-5 features	0.9231	0.9438	0.9333
CategoricalNB — WITH Laplace smoothing (α=1.0)	0.9231	0.9540	0.9222
CategoricalNB — WITHOUT smoothing (α≈0)	0.9161	0.9432	0.9222

Feature correlation check: among the top-10 selected features, 22 pairs had |correlation| > 0.85 (e.g., mean radius vs mean perimeter: r = 0.998; mean radius vs mean area: r = 0.987) — confirming these features are far from independent.

Analysis

- Effect of feature set: using all 30 features gave the best recall (97.8%) — for a medical dataset this matters, since recall reflects how many true malignant cases are caught. Reducing to 10 or 5 features traded a little recall for slightly higher precision, and training/inference became cheaper — a reasonable trade-off when features are highly redundant.
- Effect of the independence assumption: features such as 'mean radius', 'mean perimeter', and 'mean area' are almost perfectly correlated (r > 0.98) because they are geometrically derived from the same cell measurements. Naïve Bayes still performed well (~93% accuracy) despite this strong violation of its core assumption — a well-known empirical robustness of Naïve Bayes, because what matters for classification is getting the ranking of posterior probabilities right, not their exact calibrated values, and correlated features often reinforce (rather than contradict) the same signal.
- Effect of Laplace smoothing: on a deliberately small, sparse training subset, smoothing improved test accuracy (92.31% vs 91.61%) and recall stability by preventing zero-probability estimates for feature-value/class combinations that were unseen during training. Without smoothing, encountering such a combination at test time can force an entire class's posterior probability to (near) zero regardless of other evidence, causing avoidable misclassifications — this effect is most visible with limited training data, exactly as observed here.

Conclusion

Naïve Bayes is fast, effective, and reasonably robust to correlated features on this dataset, though recall (critical in medical screening) is best preserved by using more of the available features. Laplace smoothing is essential whenever the training set is small relative to the feature-value space, as it directly prevents the zero-frequency failure mode of probability estimation.

Experiment 2: MLE vs Bayesian Estimation of Distribution Parameters

Aim

To estimate the parameters (mean, variance) of a Gaussian distribution using Maximum Likelihood Estimation (MLE) and compare with a Bayesian estimation approach, observing how the prior influences results and how estimates behave under small vs large sample sizes.

Setup

Samples were drawn from a known true distribution N(μ=5.0, σ=2.0) so estimation accuracy could be measured directly. A deliberately mismatched prior N(μ_0 =2.0, τ_0 ²=1.0) was used for the Bayesian estimate, to clearly show its pull on small-sample estimates.

Theory

MLE: $\hat{\mu}$ = $(1/n)\Sigma x_i$ — the sample mean, with no prior information used.

Bayesian estimation (conjugate Normal-Normal model, known variance σ^2): the posterior mean is a precision-weighted average of the prior mean and the sample mean:

posterior mean = [(μ_0 /τ_0 ²) + ($n \cdot \bar{x}$ /σ^2)] / [($1/\tau_0$ ²) + (n/σ^2)], posterior variance = 1 / [($1/\tau_0$ ²) + (n/σ^2)]

As n grows, the data term (n/σ^2) dominates the prior term ($1/\tau_0$ ²), so the Bayesian posterior mean converges toward the MLE/sample mean, and posterior variance shrinks — the prior's influence fades with more evidence.

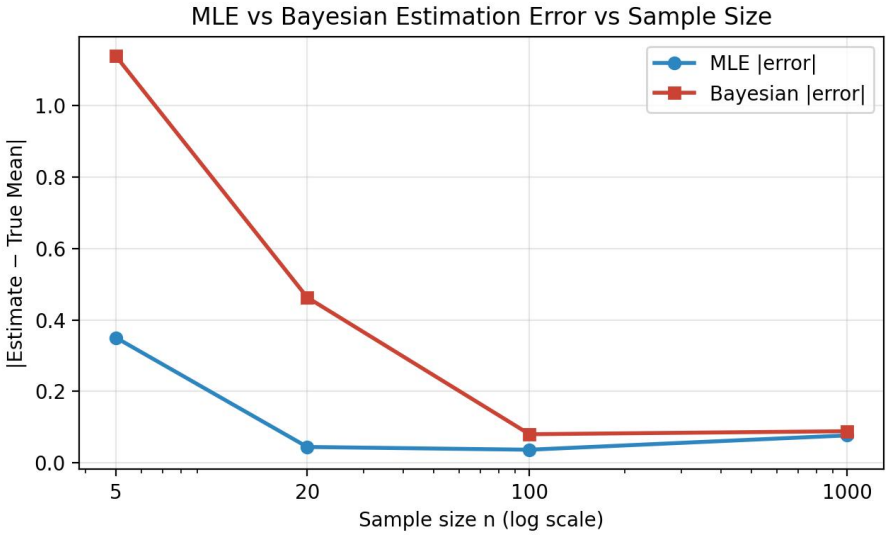
Procedure / Code

```
def bayesian_posterior(sample, mu0, tau2, sigma2):
    n = len(sample)
    sample_mean = np.mean(sample)
    post_var  = 1.0 / (1.0/tau2 + n/sigma2)
    post_mean = post_var * (mu0/tau2 + n*sample_mean/sigma2)
    return post_mean, post_var

for n in [5, 20, 100, 1000]:
    sample = np.random.normal(true_mu, true_sigma, size=n)
    mu_hat, var_hat = np.mean(sample), np.var(sample)      # MLE
    post_mean, post_var = bayesian_posterior(sample, 2.0, 1.0, sigma2)  # Bayesian
```

Results

n	MLE $\hat{\mu}$	Bayesian post. mean	Bayesian post. var.	|MLE error|	|Bayesian error|
5	5.3504	3.8613	0.44444	0.3504	1.1387
20	5.0444	4.5370	0.16667	0.0444	0.4630
100	5.0367	4.9199	0.03846	0.0367	0.0801
1000	4.9232	4.9115	0.00398	0.0768	0.0885



Analysis

- Small sample (n=5): the Bayesian estimate (3.86) is pulled strongly toward the mismatched prior (2.0), away from the true mean (5.0) — its error (1.14) is much larger than the MLE's error (0.35). With so little data, the prior dominates the posterior.
- Large sample (n=1000): the Bayesian posterior mean (4.91) and the MLE (4.92) become nearly identical, and posterior variance shrinks to 0.004 — the data has completely overwhelmed the prior, exactly as the conjugate-update formula predicts.
- Reliability implication: MLE is 'prior-free' but can be unstable/noisy with very little data (no external information to anchor it); Bayesian estimation is more stable and can incorporate genuine prior knowledge, but a poorly chosen or mismatched prior actively hurts accuracy when data is scarce. The two approaches converge as sample size grows, so the choice matters most in small-data regimes.

Conclusion

The experiment confirms the classical result that Bayesian estimates behave as a compromise between prior belief and observed data, with the data's influence growing (and the prior's shrinking) as sample size increases — while MLE, lacking any prior, is simple but can be unreliable on small samples.

Experiment 3: Bayesian Belief Network — Construction and Inference

Aim

To construct a Bayesian Belief Network (BBN) for a problem domain, perform inference under different evidence conditions, observe the effect of modifying conditional probability tables (CPTs), and evaluate how the network handles uncertainty and variable dependencies.

Problem Domain

The classical Burglary–Alarm domain (Pearl): a house alarm can be triggered by a Burglary or an Earthquake; if the alarm sounds, neighbours John and Mary may call. Network structure: Burglary → Alarm ← Earthquake; Alarm → JohnCalls; Alarm → MaryCalls.

Procedure / Code

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model = DiscreteBayesianNetwork([('Burglary','Alarm'), ('Earthquake','Alarm'),
                                 ('Alarm','JohnCalls'), ('Alarm','MaryCalls')])
# CPDs: P(Burglary)=0.001, P(Earthquake)=0.002,
# P(Alarm|B,E), P(JohnCalls|Alarm), P(MaryCalls|Alarm)  ...
model.add_cpds(cpd_burglary, cpd_earthquake, cpd_alarm, cpd_john, cpd_mary)
infer = VariableElimination(model)
infer.query(variables=['Burglary'], evidence={'JohnCalls':'True'})
```

Results — Inference Under Different Evidence

Query	P(Burglary=False)	P(Burglary=True)
No evidence (prior)	0.9990	0.0010
Evidence: JohnCalls = True	0.9837	0.0163
Evidence: JohnCalls = True, MaryCalls = True	0.7158	0.2842
Evidence: JohnCalls=True, MaryCalls=True, Earthquake=True	0.9967	0.0033

Sensitivity Experiment — Modifying the Alarm CPT

The false-alarm rate P(Alarm=True | Burglary=False, Earthquake=False) was varied, then P(Burglary=True | JohnCalls=True) was re-queried:

False-alarm rate P(Alarm=T|B=F,E=F)	P(Burglary=True | JohnCalls=True)
0.001 (original)	0.01628
0.050	0.00906
0.200	0.00385

Analysis

- Evidence accumulation: P(Burglary=True) rises from a tiny prior (0.001) to 1.63% given John's call alone, and jumps sharply to 28.4% once BOTH John and Mary call — corroborating evidence from two independent witnesses compounds strongly.
- Explaining away: adding Earthquake=True as evidence drops P(Burglary=True) from 28.4% back down to 0.33%, even though both calls are still observed. The known earthquake provides an alternative, sufficient explanation for the alarm, reducing the need to invoke a burglary — a hallmark inference pattern that only a full joint probabilistic model (not a simple rule-based system) captures correctly.
- CPT sensitivity: as the false-alarm rate is increased, calls become a progressively weaker signal for burglary (posterior falls from 1.63% to 0.39%) — because a noisy alarm is less informative about its true cause, illustrating how the reliability of the network's conclusions depends directly on the accuracy of its conditional probability tables.
- Handling uncertainty: rather than a binary yes/no diagnosis, the BBN produces a calibrated probability distribution over Burglary, letting a decision-maker weigh the evidence quantitatively and update beliefs coherently as new evidence (or new probability estimates) arrive.

Conclusion

The BBN correctly encodes and propagates the dependencies among Burglary, Earthquake, Alarm, and the two calls, produces sensible posterior updates under multiple evidence conditions, reproduces the classic 'explaining away' pattern, and demonstrates that its conclusions are only as reliable as the CPT estimates feeding it — making it both a robust and an interpretable framework for reasoning under uncertainty.

Experiment 4: Concept Learning with the Candidate-Elimination Algorithm

Aim

To implement the Candidate-Elimination algorithm and study the effect of hypothesis-space size (number of attributes) and number of training examples on sample complexity and convergence, interpreting results via the mistake-bound perspective.

Theory

Candidate-Elimination maintains the version space — all hypotheses consistent with training data seen so far — bounded by a specific boundary S and a general boundary G. For k attributes each with (up to) 2 possible values plus a wildcard '?' and a 'no value' placeholder, the size of the conjunctive hypothesis space is 3^k + 1. Larger hypothesis spaces require more training examples to converge to a unique hypothesis (higher sample complexity); the version space narrows monotonically as consistent examples arrive.

Procedure / Code (core update rules)

```
def candidate_elimination(examples, n_attrs):
    S = [['0']*n_attrs]      # most specific boundary
    G = [['?']*n_attrs]      # most general boundary
    for attrs, label in examples:
        if label == 'Yes':                  # positive example
            G = [g for g in G if consistent(g, attrs)]
            S = generalize_S_minimally(S, attrs, G)
        else:                               # negative example
            S = [s for s in S if not consistent(s, attrs)]
            G = specialize_G_minimally(G, attrs, S)
    return S, G
```

Results — 4a: Base 6-attribute EnjoySport dataset (4 examples)

Final S = { ⟨Sunny, Warm, ?, Strong, ?, ?⟩ }

Final G = { ⟨Sunny, ?, ?, ?, ?, ?⟩ , ⟨?, Warm, ?, ?, ?, ?⟩ }

This exactly matches the known textbook result for this classic dataset, validating the implementation.

Results — 4b: Hypothesis-space size vs number of attributes k

k (attributes)	|H| = 3^k + 1
2	10
4	82
6	730
8	6,562
10	59,050

Results — 4c: Version-space convergence as training examples increase

m (examples)	S boundary	G boundary
2	⟨Sunny,Warm,?,Strong,?,?⟩	⟨?,?,?,?,?,?⟩
4	⟨Sunny,Warm,?,Strong,?,?⟩	⟨Sunny,Warm,?,?,?,?⟩
6	⟨Sunny,Warm,?,Strong,?,?⟩	⟨Sunny,Warm,?,Strong,?,?⟩
8	⟨Sunny,Warm,?,Strong,?,?⟩	⟨Sunny,Warm,?,Strong,?,?⟩

Analysis

- Effect of attribute count (hypothesis-space size): the space grows exponentially (3^k + 1) with the number of attributes — from 10 hypotheses at k=2 to over 59,000 at k=10. A larger hypothesis space means more candidate hypotheses are consistent with any small set of examples, so more training examples (higher sample complexity) are required before the version space narrows enough to generalize reliably — this is the core trade-off explored by the mistake-bound model: the worst-case number of mistakes an algorithm can make is bounded by roughly $\log_2 |H|$, so an exponentially larger H directly implies a larger worst-case mistake bound.
- Effect of training set size: in the 4c experiment, G started as the fully general '?' hypothesis (m=2) and narrowed step by step until, by m=6, G had converged exactly to S — at that point the version space contains a single hypothesis, ⟨Sunny,Warm,?,Strong,?,?⟩, and the concept has been learned exactly. This demonstrates the general property that the version space shrinks monotonically (never grows) as consistent examples accumulate.

- Finite vs infinite hypothesis spaces: with a finite conjunctive hypothesis space (as here), the algorithm is guaranteed to converge to the target concept given enough noise-free, consistent examples, and its sample complexity can be bounded analytically. An infinite hypothesis space (e.g., allowing arbitrary real-valued thresholds or unrestricted boolean formulas) has no such finite bound in general — convergence and generalization then depend on other complexity measures (such as VC-dimension) rather than a simple |H| count.

Conclusion

Candidate-Elimination reliably converges to the target concept when the hypothesis space is finite and examples are consistent, but the number of examples needed to converge grows with the size (and hence complexity) of the hypothesis space — directly connecting empirical sample complexity to the theoretical mistake-bound model of concept learning.

Experiment 5: Decision Tree Classifier on a Real-World Dataset

Aim

To implement a Decision Tree classifier on a real-world dataset, train using different splitting criteria (entropy vs Gini index), evaluate using accuracy and F1-score, and analyze how tree depth and pruning affect overfitting.

Dataset

Same Wisconsin Breast Cancer dataset as Experiment 1 (569 samples, 30 features, binary target), 75/25 stratified train/test split.

Procedure / Code

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score

for criterion in ['gini', 'entropy']:
    clf = DecisionTreeClassifier(criterion=criterion, random_state=42).fit(X_train, y_train)
    # accuracy_score / f1_score on X_test

for depth in [1,2,3,4,5,6,8,10,None]:
    clf = DecisionTreeClassifier(criterion='entropy', max_depth=depth,
                                 random_state=42).fit(X_train, y_train)
    # compare train_acc vs test_acc  ->  overfitting gap

path = clf.cost_complexity_pruning_path(X_train, y_train)   # ccp_alpha values
for alpha in path.ccp_alphas:
    clf = DecisionTreeClassifier(criterion='entropy', ccp_alpha=alpha,
                                 random_state=42).fit(X_train, y_train)
```

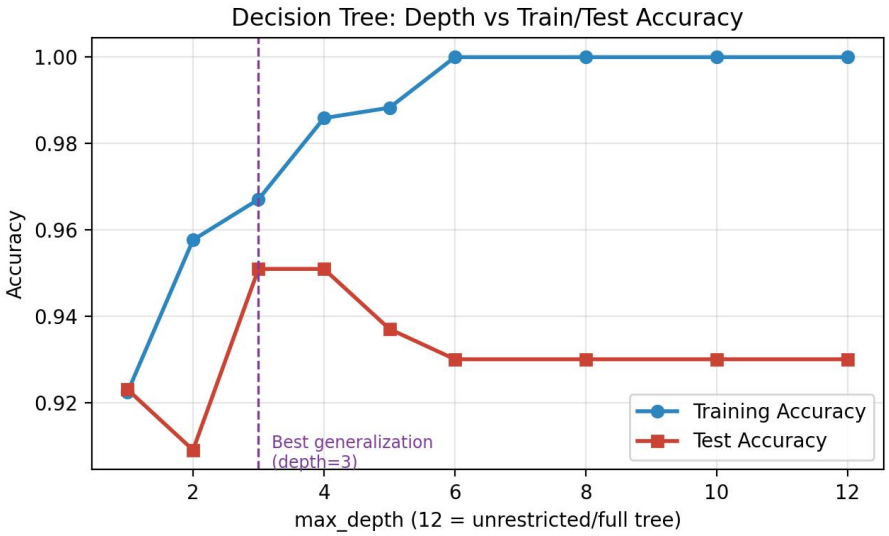
Results — Splitting Criterion Comparison

Criterion	Accuracy	F1-score	Tree Depth	Leaves
Gini index	0.9231	0.9379	7	18
Entropy (Information Gain)	0.9301	0.9432	6	15

Results — Effect of max_depth (Overfitting)

max_depth	Train Acc.	Test Acc.	Gap (overfit indicator)
1	0.9225	0.9231	−0.0005
2	0.9577	0.9091	0.0487

max_depth	Train Acc.	Test Acc.	Gap (overfit indicator)
3	0.9671	0.9510	0.0161
4	0.9859	0.9510	0.0349
5	0.9883	0.9371	0.0512
6	1.0000	0.9301	0.0699
Full (unrestricted)	1.0000	0.9301	0.0699



Results — Cost-Complexity Pruning

ccp_alpha	Train Acc.	Test Acc.	Depth	Leaves
0.00000 (unpruned)	1.0000	0.9301	6	15
0.01278	0.9859	0.9441	6	11
0.01481	0.9671	0.9510	3	7
0.02594	0.9601	0.9091	3	5
0.05537	0.9460	0.9161	2	3
0.56231 (over-pruned)	0.6268	0.6294	0	1

Analysis

- Splitting criterion: entropy (Information Gain) slightly outperformed Gini on this dataset (93.01% vs 92.31% accuracy) while also producing a smaller tree (6 vs 7 depth, 15 vs 18 leaves) — the two criteria usually give similar results since both measure node impurity, but entropy's logarithmic penalty occasionally favors marginally different splits.
- Depth vs overfitting: training accuracy climbs steadily and reaches a perfect 100% by depth 6, while test accuracy peaks around depth 3–4 (~95.1%) and then plateaus/declines slightly (93.0%) as depth increases further. The widening train–test gap (from ~0.02 at depth 3 to ~0.07 at depth 6+) is the classic signature

of overfitting: beyond a certain depth the tree starts memorising noise specific to the training set rather than learning generalizable decision rules.

- Effect of pruning: cost-complexity pruning found its best test accuracy (95.10%) at ccp_alpha≈0.0148, which corresponds to a much shallower tree (depth 3, 7 leaves) than the unpruned tree (depth 6, 15 leaves) — despite having a slightly lower training accuracy, the pruned tree generalizes better. Over-pruning (alpha=0.562) collapses the tree to a single leaf and destroys almost all predictive power (63% accuracy, essentially the majority-class baseline), showing pruning strength must be tuned, not maximized.

Conclusion

Both entropy and Gini are effective splitting criteria with similar performance; unrestricted tree growth reliably overfits this dataset, while limiting max_depth or applying cost-complexity pruning to a moderate degree (here, ~depth 3, ccp_alpha≈0.015) recovers the best generalization — confirming that controlling tree complexity, not maximizing training accuracy, is the key to a reliable decision tree model.